Tetris Undo/Redo Code Analysis

Repo: https://eae-git.eng.utah.edu/u1537288/tetris

Command Pattern Implementation

To support undo, redo, and integrate my code with Unity's Input System, I implemented a **Command Pattern–based input system** inside my TetrisInputController. Instead of having input directly modify the Piece or Board classes, every player action is wrapped in a small command object that implements a shared interface

Each input action (move, rotate, store, undo, redo, hard drop, etc.) creates a corresponding command object and enqueues it into a queue. During the game's update loop, all queued commands are executed in order on the active piece. This guarantees that input is processed deterministically and prevents multiple systems from competing to modify the game state.

This implementation follows the idea of the Command Pattern from Game Programming Patterns, but not its strict form. Instead of commands storing undo state, my commands are stateless and simply request actions. Undo and redo are handled by the board through a store-and-swap system. This avoids saving full board states, which would be expensive and would remove challenge, while still allowing meaningful undo and redo on the active piece.

```
9 references
private interface ICommand { void Execute(Piece piece); }

3 references
private sealed class MoveCommand : ICommand
{
    private readonly Vector2Int dir;
    2 references
    public MoveCommand(Vector2Int dir) => this.dir = dir;
    2 references
    public void Execute(Piece piece) => piece.Command_Move(dir);
}
```

```
1 reference
private sealed class UndoCommand : ICommand
{
    2 references
    public void Execute(Piece piece) => piece.board.Undo();
}

1 reference
private sealed class RedoCommand : ICommand
{
    2 references
    public void Execute(Piece piece) => piece.board.Redo();
}
```

Patterns I Deliberately Did Not Use

I deliberately chose not to use the **Singleton Pattern** for core systems like the Board and Piece. While singletons are common in small games, they tightly couple systems together and make code harder to test, reuse, and extend. Instead, I passed references explicitly between systems. This kept dependencies clear and made the architecture more flexible and easier to maintain.

What I Learned from Structuring the Code

The biggest lesson I learned is that design patterns must be adapted to the game, not blindly followed. Learning how to take advantage of Unity's Input System allowed me to easily add to the Commands and Input Actions, which also allows for adoption of other control methods a lot more easily, or even creating an AI to play against you. Separating input from gameplay logic also made debugging much easier. When something went wrong, I could determine whether the issue came from input, command execution, or game rules.

Catch-All / Additional Thoughts

The most challenging part of this project was designing undo and redo in a way that felt fair. At first, the system made the game too easy with the initial undo, which was to just redo the board before the piece locked. After adding the hold mechanic from the original game and through iteration, I landed on a system that allowed you to control what pieces you can undo/redo, adding more flexibility, and still put the pressure on the player in the moment, especially when they are at higher levels (faster fall speed).

The Undo/Redo needed me to add more visualization and communication to the player, as the game gets too chaotic the longer you go, adding an indication for which piece you have stored, as well as the next pieces are added to the strategy of where to place blocks, when to undo, or hold on to another piece. Also, adding score/lines cleared/level all helped give purpose and other important information, like when to expect to get a charge to store another piece.

Time Spent

I spent approximately **12 hours** on this project. Much of that time was exploring Unity's Input System and refactoring my code into a command pattern. After that, it was landing on a good way to implement the Und/Redo mechanic, making sure it was balanced, and iterating on flawed designs. Those final design choices required some extra UI elements to help communicate important information to the player, but also pushed me to add more polish to make the game more complete.